

File system Implementation

Goal: How to implement a simple file system? (as a way to introduce basic two important aspects on-disk structures, access methods and policies we would be looking at
VSFS → very simple file system)

data structures to organize data and metadata on disk

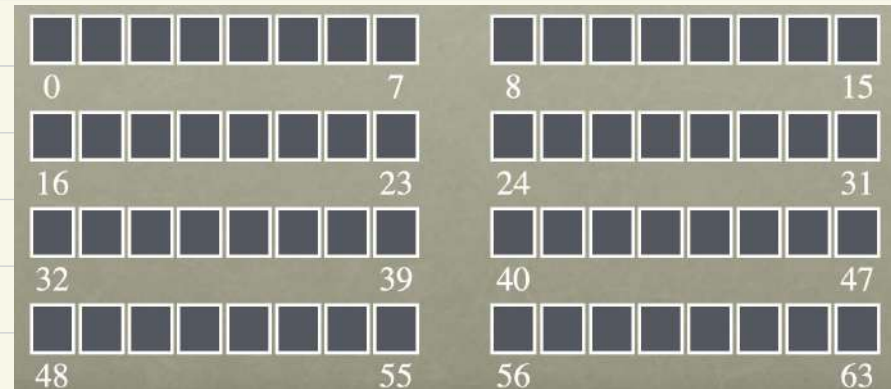
implementation of system calls like open, read, write using the data structures.

Overall organization

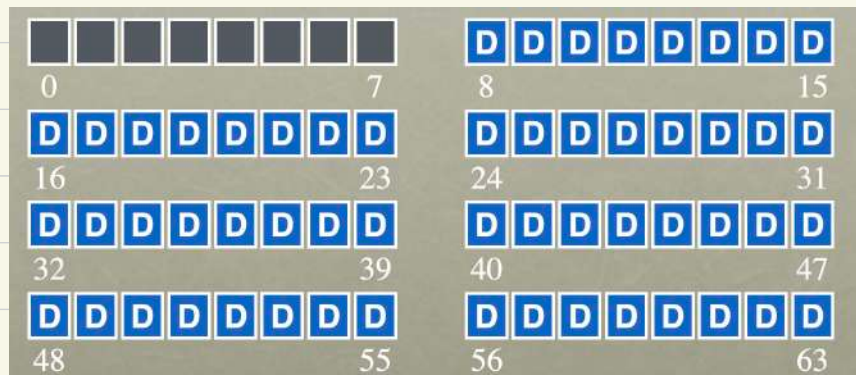
- disk is divided into blocks.
assumed of size 4 KB.
- in a partition of size N 4KB blocks, the blocks are addressed from 0 to $N-1$.

What is stored inside blocks?

→ data takes most of the space
↳ stored inside "data region".



a disk with 64 blocks, each of size 4KB $\Rightarrow$ total memory = 256KB



→ 56 out of 64 blocks become data region.

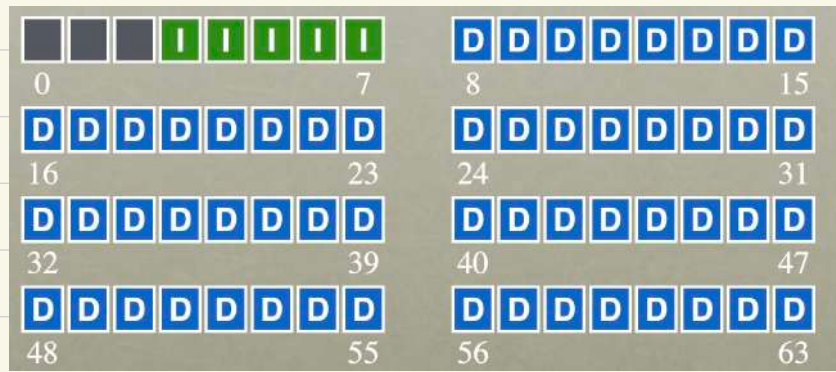
what else is stored inside disk?

metadata of files

eg. about size, data blocks location, access rights, modify times, etc.

Where is the metadata stored? → in inodes.

↳ which is further stored in a portion of disk (referred to as inode table)



→ 5 of our 64 blocks is allotted for inodes. (denoted as I)

each inode has a typical size of 128 or 256 bytes.

Assuming 256 bytes per inode, how many inodes can be stored in a 4KB block?

→ 16 inodes

type (file or dir?)
uid (owner)
rwx (permissions)
size (in bytes)
Blocks
time (access)
ctime (create)
links_count (# paths)
addrs[N] (N data blocks)

inode 16	inode 17	inode 18	inode 19
inode 20	inode 21	inode 22	inode 23
inode 24	inode 25	inode 26	inode 27
inode 28	inode 29	inode 30	inode 31

⇒ the file system in the above example can contain 80 total inodes
⇒ (in this tiny example), maximum #files we can have in our file system = 80

What else is needed? → some allocation structures

to track whether inodes or data blocks are free or allocated!

For this purpose, we have a popular structure known as bitmap.

one for data region (data bitmap) and other for inode table (inode bitmap)

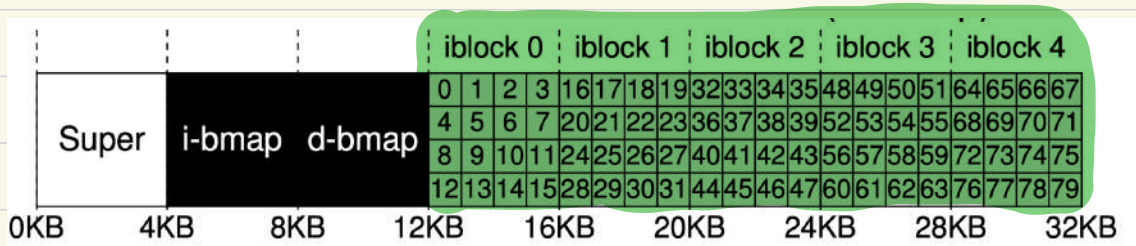
The Inode (index node)

each inode gets a number, i-number (low-level name of the file)

On a typical file system, given an i-number → we can figure out where the inode is located in the disk (inode table is indexed by inode number and we know size of an inode!)

one inode contains

metadata



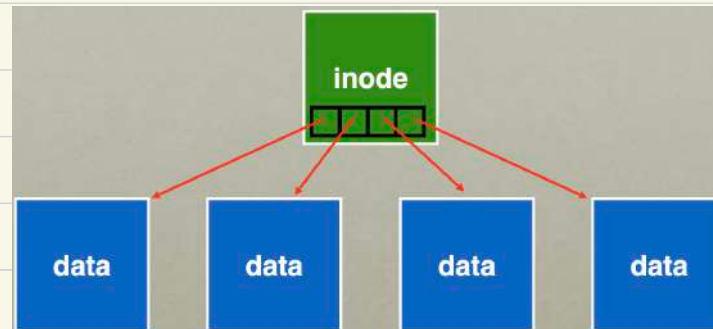
Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists

Figure 40.1: Simplified Ext2 Inode

The inode table (close up)

How does the inode refers to where data blocks are?

Simple approach
have direct pointers (disk addresses) inside the inode; each pointer refers to one disk block that belongs to the file.



Problem with this approach? limits maximum size of a stored file.

For eg:- Suppose an inode contains 12 direct pointers (like in early UNIX systems)

and each disk block is of size 4KB.

Here each data pointer is referring to one data block.

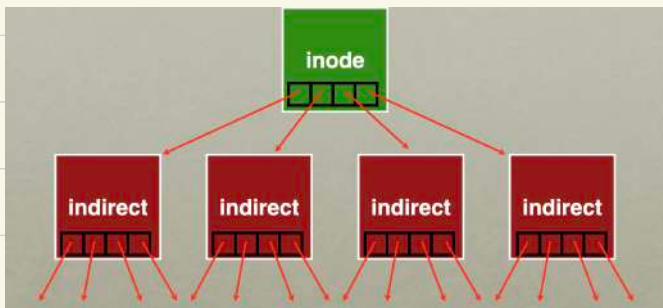
⇒ max file size = $12 \times 4 \text{ KB} = 48 \text{ KB}$.

So, imagine you have a file - say, a video file of 10MB or even a PDF of 100KB. - the inode has space to point to only 12 blocks (48KB) - once those are filled, there's no way to store the rest of the file's data because, the inode has no pointers left to reference more disk blocks. So, even though your disk has plenty of free space, the file cannot grow beyond 48KB because of the above approach!

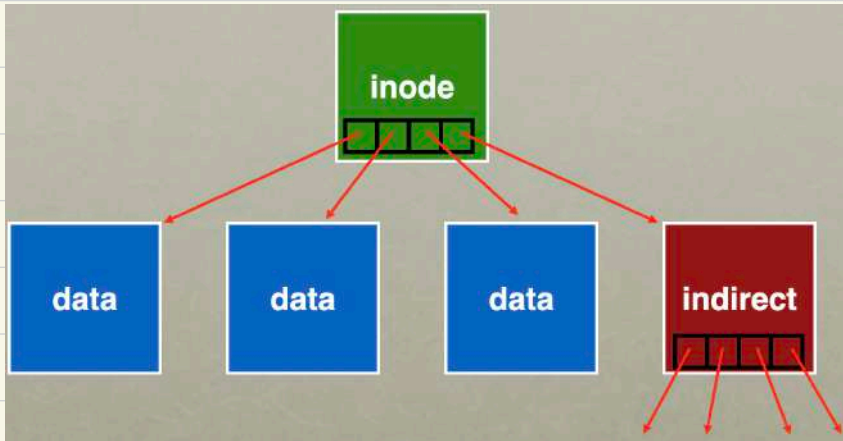
what to do? Multi-level indexing

Use an indirect pointer → instead of pointing to a block that contains user data, it points to a block that contains more pointers, each of which point to user data.

(Indirect blocks are stored in regular data blocks)



What if we want to optimize this look up for small files?



Thus, an inode may have some fixed # direct pointers and a single indirect pointer.

(an indirect block is allocated when the file grows large enough).

Suppose an inode has 12 direct entries and 1 indirect entry, what is the max. file size

1 entry $\rightarrow$ 4 byte disk address $\Rightarrow$ a 4KB sized indirect block can hold the addresses of 1K block

$\Rightarrow$ total blocks it can refer to = $12 + 1K$

$\Rightarrow$ max-file size = $(12 + 1024) \cdot 4KB = 4144 KB > 4MB!$

What if we want to support even larger files?

$\rightarrow$ use a double indirect pointer.

that refers to a block that contains pointers to indirect blocks, each of which contain pointers to data blocks.

$\Rightarrow$ can grow files with an additional $(1024 * 1024) \cdot 4KB$
 $\hookrightarrow 4GB$ in size

eg:- What is the max-file size when the inode has 12 direct pointers as well as both a single and a double indirect block.

- assuming a block size of 4KB, 4-byte pointers,
max-file size = $(12 + 1024 + 1024^2) * 4KB$

Want to increase more? → use a triple indirect pointer!

Directory organization

Contents of directory? → list of (entry name, inode number) pairs.

* A directory is a special type of file and has inode and data blocks which store the file records

For eg:- assume a directory *dir* (inode number 5) has three files in it (*foo*, *bar*, and *foobar-is-a-pretty-longname*), with inode numbers 12, 13, and 24 respectively.

Then, the on-disk data for *dir* might look like the following list.

record length
(how many bytes this directory entry occupies in the directory file)

→ the actual length of the name

inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	28	foobar-is-a-pretty-longname

→ current directory
→ parent directory

* if something gets deleted, its inode number gets modified to 0.

Note that, instead of a simple linear list directory storing can also be implemented using linked list, hash tables, binary search trees etc.

Free space management

① Using bit map:

create a file $\rightarrow$ need to allocate inode and data blocks $\rightarrow$
look at the corresponding bitmaps $\rightarrow$ update the bit map.

* additional consideration: look for a sequence of free blocks (say 8)
when a new file is created and need data blocks.
(why? as contiguous disk space of a file improves performance).

$\hookrightarrow$ pre-allocation policy.

② Free list; superblock stores pointer to first free block, a free block stores the address of next block on the list, and so on...

more complex structures can also be used.

How the file system and disk interact when a process performs a `read()` on a file?

eg:- what happens when you issue

`open("/foo/bar", O_RDONLY)` call

→ the file system first needs to find the inode for "bar".
↳ to obtain basic info about file like permissions, file size, etc.)

Question: How to find this inode using the given path name?

→ first, find the inode of the root directory (which must be well known!)

→ read the block that contains this inode number

→ find pointers to data blocks

→ read root directory data block(s)

→ find the entry for "foo"; we now have inode number of foo.

→ similarly, traverse as before to find the entry for "bar" from the data block of foo

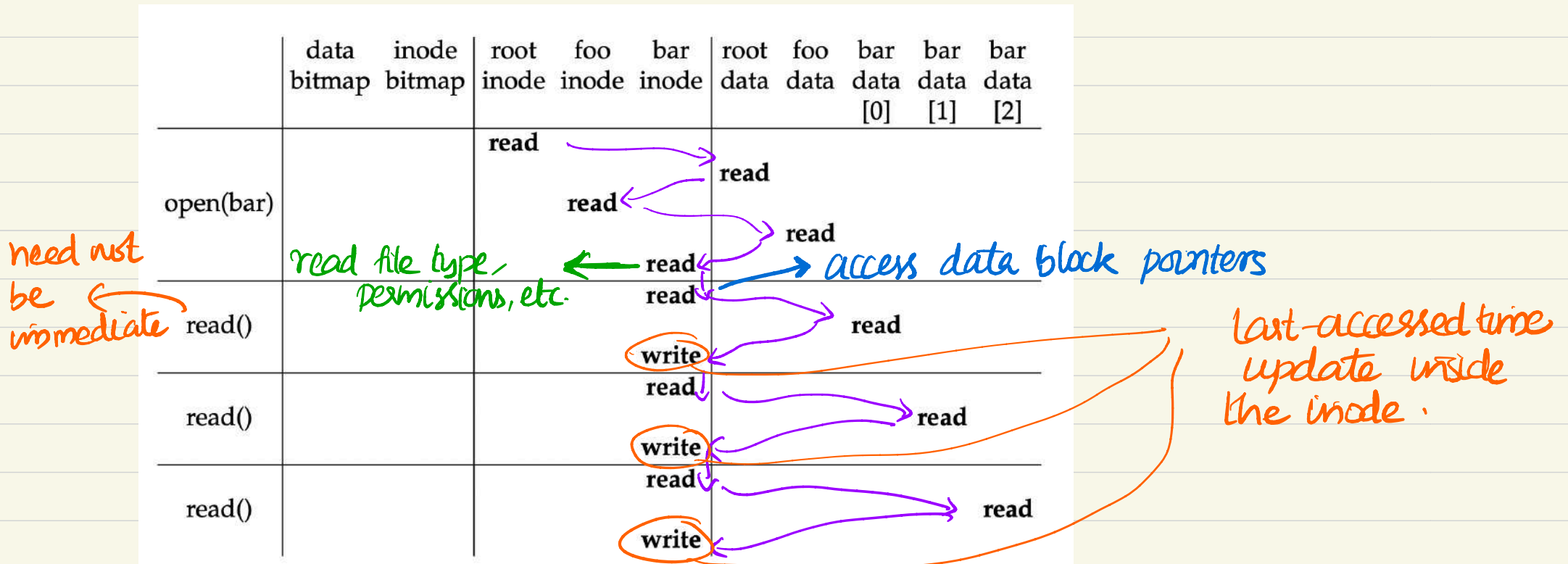
finally, we have the inode number of bar.

→ read bar's inode into memory → do permission check

→ allocates an fd for this process in the per-process open-file table

And returns it to user.

once open, the program can then issue `read()`, say, `read(fd, buf, 4096)`
 at some point, the file will be **closed** → fd deallocated
 (not much of work here!)



a summary of file read timeline in the above example.

Remark * corresponding to each directory in the path, we have to read its mode and its data.

* large directory ⇒ might have to read many data blocks to find the desired entry

writing a file to disk

- Overall idea:
- locate the parent directory (/foo)
 - allocate space for a new inode and data block
 - link the new file into the directory
 - write both metadata and actual data to disk.

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]	bar data [1]	bar data [2]
create (/foo/bar)		read write	read	read		read	read			
					read write		write			
				write						
write()	read write			read				write		
					write					
write()	read write			read						write
					write					
				read						
write()	read write				write					write

10 I/O operations might
require for create()!

} might require 5 I/O
operations per write().

a summary of file creation and writing timeline

opening/reading/writing a file incurs a huge # I/O operations, scattered over the disk.

What can a FS do to reduce the high costs of doing so many I/O?

↳ caching and buffering.

- use system memory (DRAM) to cache important blocks
eg:- recently accessed disk blocks

- future reads to the same blocks can be served directly from the memory avoiding disk I/O.

- when data is written, it's first stored in cache, then written to disk later (lazy write / delayed write)

- allows batching and reordering of disk writes improving throughput.
- risk of data loss if the system crashes in between.

Caching and buffering together tries to hide the latency of slow disk I/O and increase performance.